CMPT 383 Quiz 3 (Racket)

Summer 2026

Solution

Duration	30 minutes for the entire exam.
Coding Questions	For coding questions, use pseudocode in the style of the language the question is about.
Aids allowed	Pencil or pen, and eraser. No notes, no papers, no books, no computers, no calculators, no cheat sheets, etc.
During the exam	Raise your hand if you would like to speak with a proctor. To be fair to all students, we don't answer questions about the exam. Choose the answer based on your best understanding of the question. Re-reading the question may help.

1. Write a Racket function called `(almost-last lst)` that returns the **second to last top-level element** of `lst`. Assume `lst` is a list, and if it has length less than 2 it should return the symbol `'none`. **Don't** use any higher-order functions or inefficient functions like append. Use basic Racket functions and recursion.

Expression	Evals to
`(almost-last '())`	`'none`
`(almost-last '(a))`	`'none`
`(almost-last '(a b))`	`'a`

Expression	Evals to
`(almost-last '(a b c))`	`'b`
`(almost-last '(a b c d))`	`'c`

```
(define (almost-last lst)
 (let ([n (length lst)])
   (cond [(< n 2) 'none]
         [(= n 2) (first lst)]
         [else    (almost-last (rest lst))]
)))
```

- **1 mark** for correctly and efficiently handling n < 2 case
- **1 mark** for correctly and efficiently handling n = 2 case
- **1 mark** for correctly and efficiently handling n > 2 case
- **1 mark** for overall correct use of Racket

Okay if ending brackets incorrect, as long as format and structure looks okay.

Using `let` is **not** required.

2. Write a function called `(drop-if pred? lst)` that returns a new list that is the same `lst` except all top-level elements that satisfy `pred?` have been removed, e.g. `(drop-if even? '(1 2 3 4 5))` returns `'(1 3 5)`. You can assume `pred?` is a predicate and `lst` is list. **Implement it so the body is single call to fold-right**, and **don't** use `filter` or append in your answer.

```
(define (drop-if pred? lst)
  (fold-right (lambda (next acc)
                (if (pred? next)
                    acc
                    (cons next acc)))
              '()
              lst))
```

- **3 marks** for correct lambda function for fold-right
- **1 mark** for '() as initial value for fold-right

Okay if ending brackets incorrect, as long as format and structure looks okay.

Solution

3. For each expression, write what evaluates to in the box on the right. If an expression doesn't evaluate to anything, then write "error". Make sure all parentheses are correct. Each correct answer is worth 1 mark. Unanswered, ambiguous, empty, or unreadable responses will get 0 marks.

Expression	**What it Evaluates to**
`((first (rest (list rest first))) '(3 5 1 2))`	`3`

`(map (lambda (x) (if (list? x) x (list x)))`
`  '(a (b) c (d e))`
`)`
	`'((a) (b) (c) (d e))`

`(map (lambda (x) (cons (length x) x))` `  (map list '(a b c)))`	`'((1 a) (1 b) (1 c))`

`(filter (lambda (x) (or (symbol? x) (number? x)))`
`  (list 1 2 'three 'four '(2 3))`
`)`
	`'(1 2 three four)`

`(reverse`
` (map (lambda (x) (cons x (list x)))`
`  (reverse '(a b c))))`
	`'((a a) (b b) (c c))`

`(fold-right (lambda (x y) y)` `  0 '(1 2 3 4))`	`0`
`(fold-right equal? 3 '(3 3 3))`	`#f`
`(apply cons (reverse '((a) (2))))`	`'((2) a)`
`(M symbol?)  ;; M is the M combinator`	`#f`
`((lambda (b) ((lambda (a) (- b a)) 5)) 1)`	`-4`
`;; +c is a curried version of Racket's usual +` `(equal? (+c 2 3) 5)`	`#f`

Solution